Ejercicio 1: explicación del código de los integradores

TP4 – Dinámica molecular regida por el paso temporal

Grupo 5

1. Contexto común

El oscilador amortiguado es $m\ddot{r} = -kr - \gamma\dot{r}$ con $m = 70$ kg, $k = 10^4$ N/m, $\gamma = 100$ kg/s, $A = 1$ m, $t_f = 5$ s (`oscillator.h`). Condiciones iniciales:

$$r(0) = A, \qquad v(0) = -\frac{A\gamma}{2m},$$

que son la posición y la derivada en $t = 0$ de la solución analítica $r(t) = Ae^{-\gamma t/2m}\cos(\omega t)$.

Cada método es una función `integrate*(p, dt, steps, emit)` en `integrators.cpp`. Llama a `emit(step, r, v)` exactamente `steps+1` veces (paso 0 con el estado inicial). El motor no calcula errores ni energías: escribe (t, r, v) y el análisis se hace en Python.

La aceleración sale siempre de `oscForce(p,r,v)/p.m`, o sea $a = (-kr - \gamma v)/m$. Como la fuerza depende de la velocidad, Verlet y Velocity Verlet necesitan despejar una incógnita implícita, y los tres métodos con memoria necesitan un arranque. Esas son las adaptaciones, marcadas en recuadros naranjas.

2. Euler predictor-corrector (Teórica 4, diap. 23)

Fórmulas de la diapositiva. Predecir $v^p = v + a\Delta t$, $r^p = r + v\Delta t$; evaluar $a(t + \Delta t) = f(r^p, v^p)/m$; corregir $v = v + a(t + \Delta t)\Delta t$, $r = r + v(t + \Delta t)\Delta t$.

```
1  double r = initialPosition(p);  double v = initialVelocity(p);
2  emit(0, r, v);
3  for (long long step = 1; step <= steps; ++step) {
4      const double a  = oscForce(p, r, v) / p.m;      // a(t)
5      const double vp = v + a * dt;                   // v^p
6      const double rp = r + v * dt;                   // r^p
7      const double a1 = oscForce(p, rp, vp) / p.m;    // a(t+dt) con predichos
8      v += a1 * dt;                                   // v corregida
9      r += v * dt;                                    // r usa la v ya corregida
10     emit(step, r, v);
11 }
```

Cómo se lee. Las líneas 5–6 son el paso 1 (predecir), la 7 el paso 2 (evaluar) y las 8–9 el paso 3 (corregir). Es una traducción directa. `r += v*dt` va después de actualizar `v` porque la diapositiva corrige r con $v(t + \Delta t)$.

Adaptaciones: ninguna. Es el esquema de la diap. 23, no el Euler modificado de la diap. 10, cuya posición lleva un término $\Delta t^2 f/2m$.

3. Verlet original (diap. 13–15)

Fórmulas de la diapositiva.

$$r(t + \Delta t) = 2r(t) - r(t - \Delta t) + \frac{\Delta t^2}{m} f(t), \qquad v(t) = \frac{r(t + \Delta t) - r(t - \Delta t)}{2\Delta t}.$$

```
const double c    = p.gamma * dt / (2.0 * p.m);
const double kdt2 = p.k * dt * dt / p.m;
double r = initialPosition(p);
const double v0 = initialVelocity(p);
const double a0 = oscForce(p, r, v0) / p.m;
double rPrev = r - v0 * dt + 0.5 * a0 * dt * dt;     // r(-dt)
for (long long step = 0; step <= steps; ++step) {
    const double rNext = (2.0*r - rPrev - kdt2*r + c*rPrev) / (1.0 + c);
    emit(step, r, (rNext - rPrev) / (2.0 * dt));
    rPrev = r;
    r = rNext;
}
```

Adaptación. (1) Amortiguamiento en forma cerrada (línea 8). La fuerza es $f = -kr - \gamma v$ y en Verlet $v(t) = (r(t+\Delta t) - r(t-\Delta t))/2\Delta t$ depende de la incógnita $r(t+\Delta t)$. Sustituyendo en la fórmula de la diapositiva:

$$r_{n+1} = 2r_n - r_{n-1} - \frac{k\Delta t^2}{m} r_n - c\,(r_{n+1} - r_{n-1}), \quad c = \frac{\gamma \Delta t}{2m}.$$

Es lineal en r_{n+1}, así que se despeja: $r_{n+1}(1+c) = 2r_n - r_{n-1} - \frac{k\Delta t^2}{m} r_n + c\, r_{n-1}$. Eso es la línea 8. La diapositiva muestra fuerzas que no dependen de v, y este paso no aparece. Hace falta porque este oscilador sí tiene el término $\gamma \dot{r}$.

Adaptación. (2) Arranque (línea 6). Verlet necesita $r(-\Delta t)$, que no existe. La diap. 14 dice «Euler evaluado en $-\Delta t$» sin dar la fórmula. Se usa $r(-\Delta t) = r - v_0 \Delta t + \frac{1}{2} a_0 \Delta t^2$. El término $\frac{1}{2} a_0 \Delta t^2$ es necesario: sin él el primer paso arrastra un error de orden más bajo y el método degrada a orden 1 (lo dice el comentario del código sobre `integrateVerlet`).

Adaptación. (3) Velocidad emitida (línea 9). La velocidad es la diferencia centrada de la diap. 15 y exige $r(t+\Delta t)$, así que cada iteración calcula un paso de posición de más. Por eso el bucle va de 0 a `steps` inclusive y se obtiene v_N en el último paso. Verlet no integra v, por lo que no hay una v propia del esquema.

4. Velocity Verlet (diap. 17)

Fórmulas de la diapositiva.

$$r(t+\Delta t) = r + \Delta t\, v + \frac{\Delta t^2}{2m} f(t), \qquad v(t+\Delta t) = v + \frac{\Delta t}{2m}\big(f(t) + f(t+\Delta t)\big),$$

con los pasos intermedios $v(t+\Delta t/2) = v + a(t)\Delta t/2$ y $v(t+\Delta t) = v(t+\Delta t/2) + a(t+\Delta t)\Delta t/2$.

```
const double c      = p.gamma * dt / (2.0 * p.m);
const double kdt2m  = p.k * dt / (2.0 * p.m);
double r = initialPosition(p);  double v = initialVelocity(p);
double a = oscForce(p, r, v) / p.m;
emit(0, r, v);
for (long long step = 1; step <= steps; ++step) {
    const double vHalf = v + 0.5 * a * dt;          // paso intermedio 1
    r += vHalf * dt;                                // r(t+dt)
    v = (vHalf - kdt2m * r) / (1.0 + c);            // paso intermedio 2
    a = oscForce(p, r, v) / p.m;                    // a(t+dt), se reutiliza
    emit(step, r, v);
}
```

Errata de la teórica. La diapositiva imprime el término de posición como $\Delta t^2 f/m$. El correcto es $\Delta t^2 f/(2m)$. Sale del Taylor de la diap. 15 y es lo que da `r += vHalf*dt`, porque $\Delta t(v+\frac{1}{2}a\Delta t) = v\Delta t + \frac{1}{2}a\Delta t^2$. Con $\Delta t^2 f/m$ el método colapsa a orden 2.

Adaptación. Segundo medio paso implícito (línea 9). El paso intermedio 2 necesita $a(t+\Delta t) = (-kr_{n+1} - \gamma v_{n+1})/m$, que depende de la incógnita v_{n+1}. Escribiendo $v_{n+1} = v_{half} + \frac{\Delta t}{2} a_{n+1}$ y despejando:

$$v_{n+1}\Big(1 + \frac{\gamma \Delta t}{2m}\Big) = v_{half} - \frac{k\Delta t}{2m} r_{n+1}.$$

La línea 9 es esa ecuación. Se resuelve exacta porque la fuerza es lineal en v, y la diapositiva no cubre fuerzas con v. Después se recalcula `a` con la v nueva (línea 10), de modo que el paso siguiente la use como $a(t)$.

5. Beeman predictor-corrector (diap. 20)

Fórmulas de la diapositiva.

$$\begin{aligned} r(t+\Delta t) &= r + v\Delta t + \tfrac{2}{3}a(t)\Delta t^2 - \tfrac{1}{6}a(t-\Delta t)\Delta t^2, \\ v^{pred} &= v + \tfrac{3}{2}a(t)\Delta t - \tfrac{1}{2}a(t-\Delta t)\Delta t, \\ v^{corr} &= v + \tfrac{1}{3}a(t+\Delta t)\Delta t + \tfrac{5}{6}a(t)\Delta t - \tfrac{1}{6}a(t-\Delta t)\Delta t, \end{aligned}$$

con $a(t+\Delta t)$ evaluada con $r(t+\Delta t)$ y v^{pred}.

```
1  double a = oscForce(p, r, v) / p.m;
2  const double rBack = r - v * dt + 0.5 * a * dt * dt;
3  const double vBack = v - a * dt;
4  double aPrev = oscForce(p, rBack, vBack) / p.m;      // a(-dt)
5  emit(0, r, v);
6  for (long long step = 1; step <= steps; ++step) {
7      const double rNew  = r + v*dt + (2.0/3.0)*a*dt*dt - (1.0/6.0)*aPrev*dt*dt;
8      const double vPred = v + (3.0/2.0)*a*dt - (1.0/2.0)*aPrev*dt;
9      const double aNew  = oscForce(p, rNew, vPred) / p.m;
10     const double vNew  = v + (1.0/3.0)*aNew*dt + (5.0/6.0)*a*dt - (1.0/6.0)*aPrev*dt;
11     aPrev = a;  a = aNew;  r = rNew;  v = vNew;
12     emit(step, r, v);
13 }
```

Cómo se lee. Las líneas 7–10 son, una a una, las cuatro ecuaciones de la diapositiva. La línea 11 desplaza la memoria: $a(t) \to a(t-\Delta t)$ y $a(t+\Delta t) \to a(t)$.

Adaptación. (1) Arranque (líneas 1–4). Beeman necesita $a(-\Delta t)$. La diapositiva no dice cómo conseguirlo. Se toma un estado Euler hacia atrás ($r(-\Delta t) = r - v\Delta t + \frac{1}{2}a\Delta t^2$, $v(-\Delta t) = v - a\Delta t$) y se evalúa la fuerza ahí.

Adaptación. (2) Una sola evaluación de fuerza por paso (líneas 9 y 11). $a(t+\Delta t)$ se calcula una vez con (r_{new}, v_{pred}), como pide la diapositiva, y se guarda para usarla como $a(t)$ en el paso siguiente. No hay una evaluación extra con la v corregida. Por eso Beeman no necesita despejar ninguna ecuación implícita: el predictor de v ya la evita.

6. Resumen de adaptaciones

Método	Adaptación	Motivo
Euler PC	ninguna	Directo de la diap. 23.
Verlet	r_{n+1} en forma cerrada	La fuerza depende de v, que en Verlet es centrada e implica r_{n+1}.
Verlet	$r(-\Delta t)$ con $\frac{1}{2}a_0\Delta t^2$	Falta el paso previo; sin ese término degrada a orden 1.
Verlet	v por diferencia centrada, un paso extra	Es lo que dice la diap. 15; exige $r(t+\Delta t)$.
Velocity Verlet	$\Delta t^2/2m$ en vez de $\Delta t^2/m$	Errata de la diapositiva.
Velocity Verlet	v_{n+1} en forma cerrada	$a(t+\Delta t)$ depende de $v(t+\Delta t)$.
Beeman	$a(-\Delta t)$ con estado Euler hacia atrás	La diapositiva no da el arranque.